



UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

Nombre Del Alumno:

Alejandro Contreras Martinez

Emmanuel Castro Salvador

Carlos Eduardo Hernandez

Camacho

Danna Paola Tobón Mosqueda

Manelic Alitzel Reyes Torres

Docente:

José Miguel Carrera Pacheco

**Login seguro, accesible y
controlado por teclado**

**ING EN DESARROLLO
DE SOFTWARE
MULTIPLATAFORMA**

Login funcional por teclado (video).

https://drive.google.com/file/d/1_LC4SjonUYiJmKD8rdiZ9IAmqyljrEVL/view?usp=sharing

Validaciones accesibles

The image shows two side-by-side screenshots of the 'Crear cuenta' (Create account) form on the UTPL website. The form is titled 'Crear cuenta' and has a 'Inicio / Registro' link. It contains several input fields: 'Nombre' (Name) with the value 'felipe', 'Apellido' (Last name) with the value 'torres', 'Matrícula' (ID number) with the value '3515445644', 'Teléfono' (Phone number) with the value '2382244889', 'Correo electrónico' (Email) with the value 'felipe@gmail.com', and 'Contraseña' (Password). The 'Matrícula' and 'Contraseña' fields are highlighted with a red border and a red error message: 'La matrícula es obligatoria' and 'La contraseña es obligatoria' respectively. Below the fields, there is a green button labeled 'Crear cuenta'.

The image shows a single screenshot of the 'Crear cuenta' (Create account) form on the UTPL website. The form is titled 'Crear cuenta' and has a 'Inicio / Registro' link. It contains several input fields: 'Nombre' (Name) with the value 'felipe', 'Apellido' (Last name) with the value 'torres', 'Matrícula' (ID number) with the value '3515445644', 'Teléfono' (Phone number) with the value '2382244889', 'Correo electrónico' (Email) with the value 'felipe@gmail.com', and 'Contraseña' (Password). The 'Contraseña' field is highlighted with a red border and a red error message: 'La contraseña es obligatoria'. Below the fields, there is a green button labeled 'Crear cuenta'.

las validaciones accesibles aseguran que cuando comete un error en un formulario (por ejemplo, dejar un campo vacío escribir mal un correo).

1. Detecte el error.
2. Muestre un mensaje claro y entendible.
3. Indique exactamente en qué campo está el problema.
4. Sea usable con teclado y lectores de pantalla.
5. No dependa solo de colores para indicar errores.

En las imágenes anteriores se muestra como el sistema cumple con una validación accesible ya que muestra un mensaje cuando un campo se deja vacío.

Manejo de sesión por rol.

Cuando un usuario inicia sesión, el sistema crea una sesión (o token) que guarda información del usuario, como su ID, nombre y rol.

El rol indica el nivel de permisos que tiene dentro de la aplicación.

Gracias a esto, el sistema puede mostrar diferentes funciones o restringir accesos según el tipo de usuario.

```
7 CREATE TYPE user_role AS ENUM ('usuario', 'admin');  
8 END IF;
```

Aquí se define un tipo de dato personalizado llamado user_role

Los valores son:

- usuario → usuario normal del sistema
- admin → usuario con permisos de administrador

```
role user_role NOT NULL,
```

Vemos que cada registro tiene un rol asignado

```

INSERT INTO users (full_name, email, role, password)
VALUES
  ('Admin SGIM', 'admin@sgim.com', 'admin', '$2b$10$Ca15N41RKjcpJvCRPGBYQ.SFTa5ywm/exymgcww.GkHf4d
  ('Usuario SGIM', 'user@sgim.com', 'usuario', '$2b$10$Iob/X7pck5pgN6amwOowGeb2eeQKhivPQrLnsXPktuG
ON CONFLICT (email) DO NOTHING;

```

Esto permite que al probar el login, puedas verificar:

- admin@sgim.com → acceso a panel de administración
- user@sgim.com → acceso al panel de usuario normal

Tests backend.

Endpoint	Método	Token	Role requerido	Status esperado	Resultado esperado
/health	GET	No	N/A	200	{ ok: true }
/auth/login	POST	No	N/A	200	{ token }
/me	GET	Sí	N/A	200	{ user }
/me	GET	No	N/A	401	Unauthorized
/admin/reports	GET	Sí	admin	200	Reportes
/admin/reports	GET	Sí	user	403	Forbidden
/no-existe	GET	N/A	N/A	404	{ error: "not_found", message, path }
/test/boom	GET	N/A	N/A	500	{ error: "internal_error", message, traceId }

```
at: server:app (node_modules/express/lib/express.js:481:7)
PASS tests/jwt-access.test.js (5.379 s)
  JWT access control (public/private/roles)
    ✓ Public: GET /health -> 200 (162 ms)
    ✓ Public: POST /auth/login -> 200 returns token (173 ms)
    ✓ Private: GET /me sin token -> 401 (28 ms)
    ✓ Private: GET /me con token -> 200 (72 ms)
    ✓ Role: GET /admin/reports con user -> 403 (111 ms)
    ✓ Role: GET /admin/reports con admin -> 200 (102 ms)
    ✓ 404: ruta inexistente -> 404 (47 ms)
    ✓ 500: ruta boom -> 500 (536 ms)

Test Suites: 2 passed, 2 total
Tests: 11 passed, 11 total
Snapshots: 0 total
Time: 7.385 s
```

Todos los tests de backend pasaron correctamente.

Estado general

Test Suites: se tiene 2 archivos de tests (por ejemplo routes.test.js y tu otro archivo de JWT/roles). Ambos pasaron.

Tests: se ejecutaron 11 tests individuales, y todos pasaron.

Detalle del test routes.test.js:

Route tests (200 / 404 / 500)

GET	/health	->	200	(105 ms)	
GET	/ruta-que-no-existe	->	404 con JSON usable	(25 ms)	GET
/__test__	/boom	->	500 con traceld	(59 ms)	

GET /health: la API responde correctamente.

GET /ruta-que-no-existe: pasó la app maneja rutas inexistentes con JSON adecuado.

GET /test/boom: los errores internos devuelven traceld como esperado.

Ademas de que se realizaron diferentes pruebas en postman para validar la funcionalidad de las rutas e información que nos llega a través de ellas

